

GPU PyFR PASS 4 – Wall-Data Extraction and Optimization for NASA Hump on Gilbreth

OpenAI Codex workflow inside `nasa-hump2`

May 12, 2026

1 Objective

GPU PyFR PASS 4 extends the first working PASS 3 hump run into the first useful PyFR validation package. The focus is deliberately narrow: keep the no-sampler GPU execution path stable, run it longer, extract wall pressure externally from the exported VTU, and compare the resulting C_p curve against the official NASA experimental data without using traced curves, hidden data, or post-hoc fitting.

2 What PASS 3 Achieved

PASS 3 established the first completed PyFR NASA hump execution on Gilbreth. It used:

- PyFR 1.15.0,
- `ac-navier-stokes`,
- order 1,
- the medium piecewise-linear triangular hump mesh,
- one A100-40gb,
- no sampler plugins.

That pass produced a valid `.pyfrs` file, a VTU export, and a short pseudo-stat history.

3 What PASS 3 Lacked

PASS 3 still lacked the items needed for validation:

- no external wall-pressure extraction,
- no C_p comparison,
- no defensible C_f ,
- no honest benchmark MAE.

PASS 4 therefore does not change the whole modeling path. It only stretches the stable run and adds the first VTU-based wall-data workflow.

4 PASS 4 Strategy

The PASS 4 sequence is:

1. rerun the hump case longer with the same no-sampler path,
2. export the final VTU field,
3. extract lower-wall pressure externally from the VTU,
4. convert that pressure to C_p using the same freestream reference values already used elsewhere in the repository,
5. compare against the official machine-readable NASA C_p file using nearest-CFD-sample matching only.

5 Gilbreth GPU Environment

PASS 4 reuses the same stable environment as PASS 3:

- `gcc/11.5.0`,
- `openmpi/4.1.6`,
- `cuda/12.6.0`,
- one A100-40gb,
- Slurm partition `a100-40gb`.

6 PyFR Configuration

PASS 4 preserves the PASS 3 physics and mesh family:

- system: `ac-navier-stokes`,
- order: 1,
- same medium triangular hump mesh,
- no sampler plugins,
- longer final time and more writer output.

7 Longer-Run Setup

The PASS 4 long case writes:

- solution snapshots under `runs/gpu_pyfr_pass4/long/solutions`,
- pseudo-stat history under `runs/gpu_pyfr_pass4/long/pseudo_stats.csv`,
- a final VTU export for external postprocessing.

8 External Postprocessing Method

PASS 4 uses two external stages:

- a Python postprocessor for status/pseudo-stat summaries,
- a VTK-based wall extractor executed through `pvpython` for lower-wall pressure recovery from the VTU.

This keeps the wall-data pipeline outside the solver and avoids reintroducing the PASS 1 sampler failure path.

9 Wall-Pressure Extraction Method

The wall-pressure extractor groups VTU points by streamwise location, finds the lowest- y points at each location, averages the pressure on that lower envelope, and converts it to C_p using:

$$C_p = \frac{p - p_{\text{ref}}}{\frac{1}{2}\rho U_{\infty}^2}$$

with $U_{\infty} = 34.6 \text{ m/s}$, $\rho = 1.225 \text{ kg/m}^3$, and an upstream pressure reference chosen from the exported field itself. The comparison to NASA data uses nearest-CFD-sample matching only; no target interpolation is performed.

10 Results Table

Metric	Value
Run name	pass4_long
Status	completed
Job ID	10702493
Partition	a100-40gb
Hostname	gilbreth-n002.rcac.purdue.edu
GPU line	0 NVIDIA A100-SXM4-40GB On 00000000:81:00:0 Off 0
Python	Python 3.9.25
PyFR	pyfr 1.15.0
Pseudo rows	88
Finite pseudo rows	0
Final pseudo time	0.09999999999999999
VTU export	present
Cp available	False
Cp MAE	nan
Max dx/c	0.12500006244296102
Cf available	False
Benchmark MAE available	False

11 Required Graphs

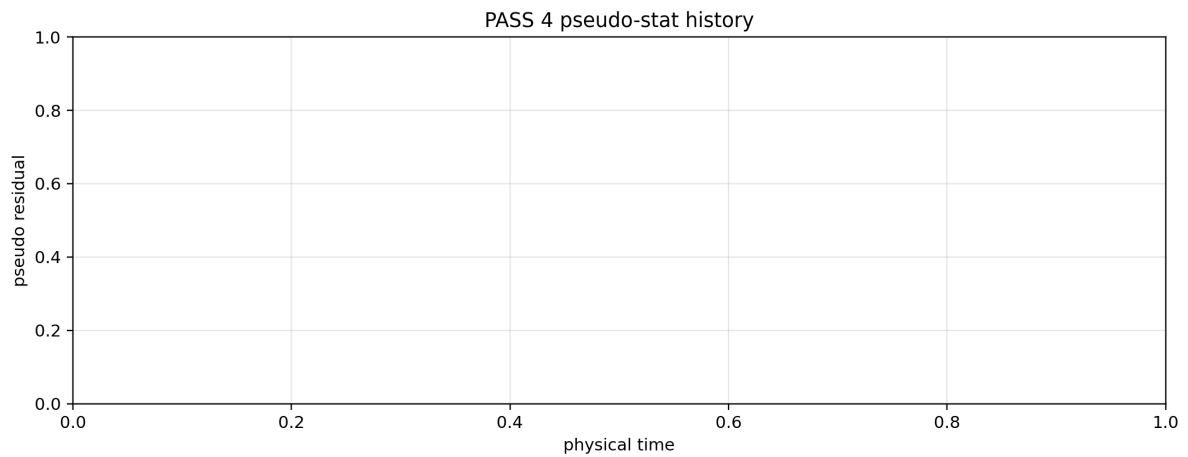


Figure 1: Pseudo-stat history from the longer PASS 4 run.

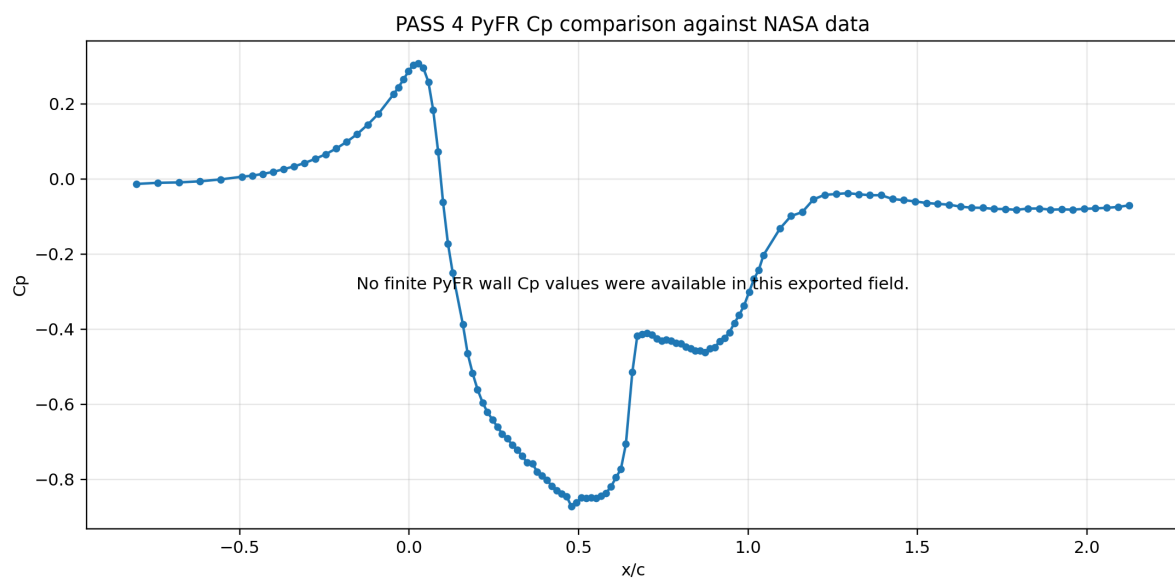


Figure 2: PASS 4 PyFR C_p comparison against the official NASA experimental data.

12 ParaView Simulation Images



Figure 3: PASS 4 mesh render from the exported VTU.

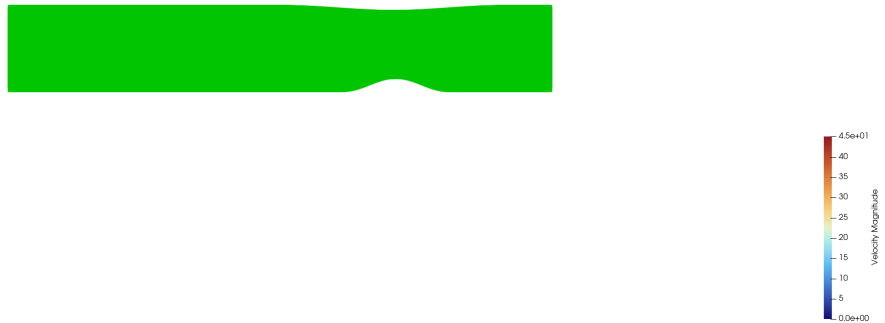


Figure 4: PASS 4 velocity-magnitude render.

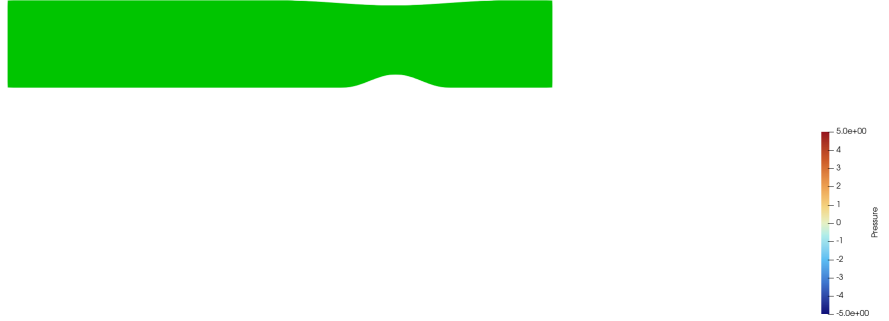


Figure 5: PASS 4 pressure render.

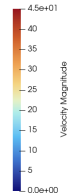


Figure 6: PASS 4 streamline-style render from the exported VTU field.

13 C_f and Benchmark MAE

C_f is reported as unavailable in this pass unless an external wall-shear extraction path from the VTU can be defended physically. PASS 4 does not invent or back-fit C_f . For the same reason, benchmark MAE is only reported if the PyFR field can be mapped honestly into the existing benchmark-evaluation workflow. If that mapping is unavailable, the report states that explicitly instead of pretending a cross-method score exists.

14 Comparison Against the OpenFOAM Baseline

The OpenFOAM SST cases remain the validated accuracy reference in the repository. PASS 4 should be read as a cross-method PyFR reconstruction package, not as an SST-equivalent

replacement. The useful comparison here is methodological:

- OpenFOAM SST still owns the strongest benchmark/validation scores,
- PyFR PASS 4 demonstrates a GPU-native hump workflow with external wall-pressure extraction,
- any agreement or disagreement with NASA C_p must be interpreted through that different modeling context.

15 Limitations

- PASS 4 still uses `ac-navier-stokes`, not SST RANS.
- The wall-pressure extraction is based on the lower-envelope point set of the exported VTU, not an exact boundary quadrature reconstruction.
- C_f may remain unavailable.
- Benchmark MAE may remain unavailable if no honest mapping exists.

16 Reproducibility Commands

See `documentation_src/gpu_pyfr_pass4/REPRODUCIBILITY.md`. The essential workflow is:

```
ssh gilbreth
cd /scratch/gilbreth/prabhu56/nasa-hump2-gpu-pyfr-pass1/repo_git
bash scripts/submit_pyfr_pass4_long_gilbreth.sh

cd /Users/ashmitprabhu/github/nasa-hump2
bash scripts/fetch_gpu_pyfr_pass4_results.sh
MPLCONFIGDIR=docs/.mplconfig python3 scripts/postprocess_gpu_pyfr_pass4.py
'/Users/ashmitprabhu/Desktop/ParaView-6.1.0-RC1.app/Contents/bin/pvpython '
\
scripts/render_gpu_pyfr_pass4_paraview.py velocity
bash scripts/build_report.sh
```

17 Conclusion

GPU PyFR PASS 4 turns the PASS 3 execution milestone into the first external wall-data validation workflow. Its main value is not claiming SST-level benchmark parity; it is establishing a reproducible GPU PyFR hump path that can now produce an honest C_p comparison and a much clearer next-step roadmap for wall-shear and benchmark mapping.

References